Lecture 19

Linked Lists

- Each item in the list part of a structure that also contains a link to the structure containing the next item. This type of list is called a linked list because it is a list whose order is given by links from one item to next item.
- Each structure of the list is called node and consist of two fields, one containing the item, and the other containing the address of the next item (A pointer to the next item) in the list.
- The link is in the form of a pointer to another structure of the same type. Such structure is represented as follows:

```
struct node{

      int item;

      struct node *next;

}
```

continue…

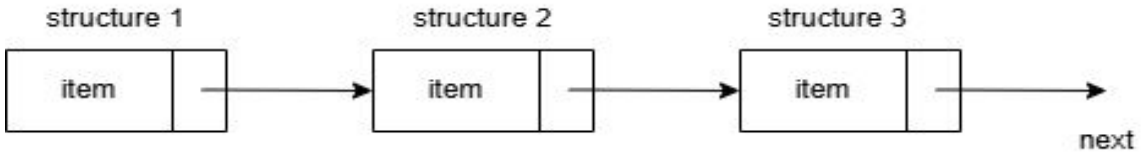
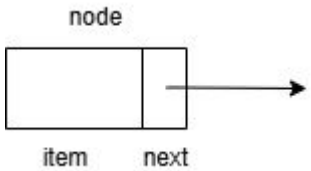


Fig: A linked list

- The first member is an integer item and the second a pointer to the next node in the list as shown below. Here the item is an integer only for simplicity, and could be any complex data type.

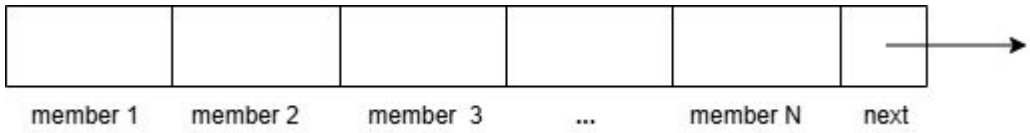


continue…

- A node may be represented in general form as follows:

```
struct tag-name{
      datatype member1;
      datatype member2;
      …
      …
      struct tag-name *next;
}
```

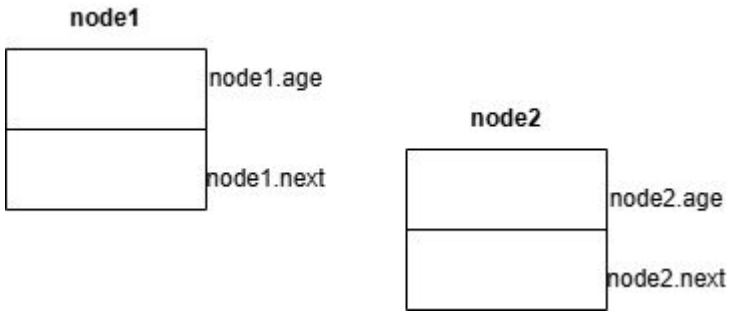
- The structure may contain more than one item with different data types. However one of the items must be a pointer of the type tag-name.



continue…

- Let us consider a simple example to illustrate the concept of linking. Let's define the structure as follows:

```
struct link_list{
      float age;
      struct link_list *next;
};
struct link_list node1, node2;
```

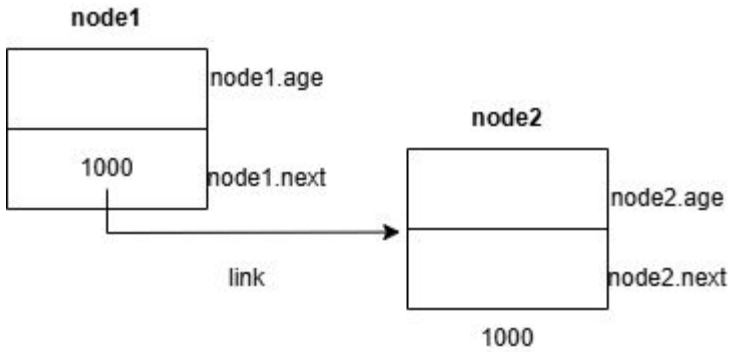


continue…

- The **next** pointer of **node1** can be made to point to **node2** by the statement.

  `node1.next = &node2;`

- **node1.next** establishes a **link** between **node1** and **node2**.

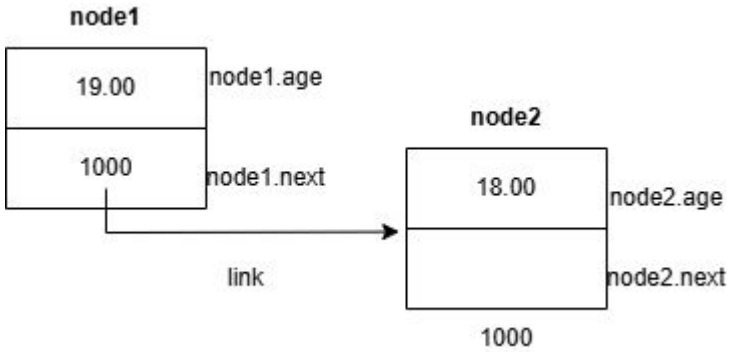


continue…

- Let us assign values to the field age.

```
node1.age = 19.00;

node2.age = 18.00;
```

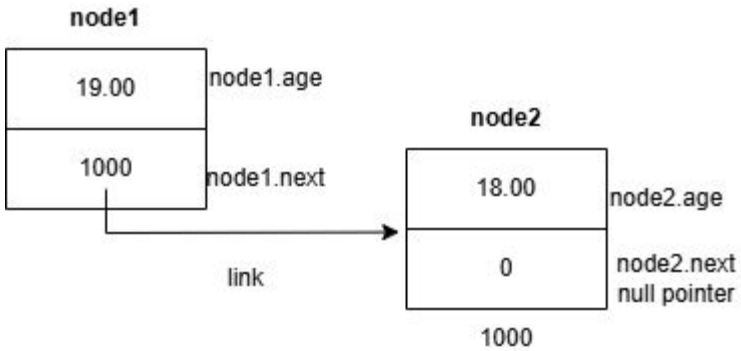


- We may continue this process to create a linked list of any number of values.

continue…

- Every list must have an end. Therefore we must indicate the end of a linked list. This is necessary for processing the list.
- C has special pointer value called **null** that can be stored in the **next** field of the last node.

```
node2.next = 0;
```



continue…

- The value of the age member of **node2** can be accessed using the **next** member of **node1** as follows:

```
printf(“%f\n”, node1.next -> age);
```